1 Abstract

This report summarizes the data integrity assessment and quantitative analysis of HPLC chromatographic data from a multi-stage purification process. The primary objective was to evaluate the robustness of UV signal normalization strategies and determine the feasibility of using available concentration data for proxy quantification. Due to severe data quality issues, including ¿65% missing values in critical concentration variables (‘Conc_B_ml‘) and high process variance in UV signals (standard deviation ¿ 390 mAU), the analysis focused on filtering anomalies and assessing the validity of linear regression models. The investigation revealed that static baseline correction is insufficient to mitigate process instability. Furthermore, an attempt to construct a calibration curve using available ‘Conc_B_%‘ data against ‘UV_1_280_mAU‘ signals resulted in a statistically insignificant linear relationship (R-squared = 0.000157). Consequently, the current dataset does not support quantitative proxy quantification without first addressing the underlying data quality and variance issues.

2 Introduction

The analysis of chromatographic data from a multi-stage purification process presents significant challenges regarding data integrity and quantitative interpretation. The dataset, comprising variables such as UV absorbance at 280 nm and 260 nm, flow rates, and conductivity, exhibits high variance in UV signals that may stem from either instrument noise or process drift. A critical constraint in this study is the severe missingness of direct concentration data (‘Conc_B_ml‘), which is absent in over 65% of records. To address this, the analysis plan involved three primary steps: (1) data integrity filtering to exclude unreliable metadata and physical anomalies like negative volumes; (2) evaluation of normalization strategies to determine if dynamic baseline adjustment is superior to static correction; and (3) construction of a calibration curve using limited available concentration data (‘Conc_B_%‘) to establish a response factor for proxy quantification. This report details the motivation for these steps, the methodology employed to assess data quality, and the critical findings regarding the validity of the proposed quantitative models.

3 Methodology

The data analysis workflow was structured into three distinct phases to ensure rigorous evaluation of the chromatographic data. First, a data integrity and anomaly filtering phase was conducted on the ‘chromatography_combined.csv‘ dataset. Rows were explicitly filtered to exclude stages labeled 'Preparation' or 'Cleanup', as well as samples marked as 'Blank' or 'Control'. Additionally, any records with a negative ‘volume_ml‘ were removed to eliminate physical anomalies. The Signal-to-Noise Ratio (SNR) for UV signals was calculated using the standard deviation of the signal divided by the Median Absolute Deviation (MAD) of the signal during the stable baseline period (the first 10% of the run volume). Second, a normalization strategy evaluation was performed. Two methods were compared: static baseline correction (subtracting the mean of the run) and dynamic normalization scaling UV signals by the mean ‘System_flow_ml_min‘ for each specific run. Missing ‘pH_ml‘ values were excluded from these calculations. Third, a calibration curve was constructed to enable proxy quantification. Given the high missingness of ‘Conc_B_ml‘, the analysis utilized available ‘Conc_B_%‘ data. A linear regression model was fitted between the mean ‘UV_1_280_mAU‘ signal and ‘Conc_B_%‘ to determine the response factor (slope) and assess the statistical significance of the relationship via R-squared and p-values.

4 Results and Discussion

The initial data integrity checks identified significant process instability, with UV signal standard deviations exceeding 390 mAU, suggesting that high variance is likely driven by process noise rather than instrument drift. While the normalization strategy evaluation aimed to distinguish between static and dynamic correction methods, the primary focus of the quantitative assessment was the calibration curve. As illustrated in Figure 1, the linear regression model fitted to the available ‘Conc_B_%‘ data yielded the equation y =

0.003925x + 14.515409. However, the analysis of the model's fit revealed a critical flaw: the R-squared value was calculated at 0.000157. This value indicates an almost non-existent linear relationship between the UV signal and the concentration variable, rendering the model statistically useless for proxy quantification. Despite the reported p-value suggesting statistical significance, the negligible R-squared value contradicts this conclusion, implying that the UV signal variance is dominated by factors other than concentration. The Standard Error of the Estimate (0.000507) is negligible compared to the signal magnitude, further confirming that the model fits the noise rather than the signal. Consequently, the conclusion that the relationship is suitable for proxy quantification is invalid. The data does not support the use of UV signals for quantitative estimation without first addressing the severe data quality issues and high variance identified in the broader dataset context.

5 Conclusion

The comprehensive data analysis of the HPLC chromatographic dataset highlights substantial barriers to robust quantitative interpretation. The high variance in UV signals, driven by process instability rather than instrument noise, necessitates dynamic normalization strategies over static baseline correction. However, the most critical finding relates to the feasibility of proxy quantification. The attempt to establish a calibration curve using available concentration data resulted in a statistically insignificant linear relationship (R-squared = 0.000157). This indicates that the current UV signals cannot reliably quantify the concentration of the analyte. Therefore, the proposed quantitative interpretation method is currently invalid. Future work must prioritize resolving the data quality issues, specifically the high process variance and the severe missingness in concentration data, before a valid response factor can be established for proxy quantification.

A Appendix

A.1 A: Data Analysis Output Figures

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
import os

def run_analysis():
    # Define file paths
    input_path = '/inputs/chromatography_combined.csv'
    output_md_path = '/workspace/data_integrity_report.md'

    # Load dataset
    df = pd.read_csv(input_path)

    # Filter 1: Exclude rows where chromatography_stage is 'Preparation' or '
        Cleanup'
    # AND Sample_Code is 'Blank' or 'Control'.
    mask_exclude_stage = df['chromatography_stage'].isin(['Preparation', 'Cleanup'
        ])
    mask_exclude_sample = df['Sample_Code'].isin(['Blank', 'Control'])

    # Apply filters
    df_filtered = df.copy()
    df_filtered = df_filtered[~mask_exclude_stage]
    df_filtered = df_filtered[~mask_exclude_sample]

    # Filter 2: volume_ml >= 0
```

```
df_filtered = df_filtered[df_filtered['volume_ml'] >= 0]

# Count of rows filtered due to negative volume
negative_volume_count = len(df) - len(df_filtered)

# Pre-sort by volume_ml once to avoid repeated sorting costs
df_sorted = df_filtered.sort_values('volume_ml').reset_index(drop=True)

# Calculate SNR using vectorized operations
def calculate_snr_group(group):
    group = group.sort_values('volume_ml').reset_index(drop=True)
    n_rows = len(group)
    if n_rows == 0:
        return 0.0

    baseline_count = max(1, int(n_rows * 0.1))
    baseline_data = group.iloc[:baseline_count, group.columns.get_loc(['
        UV_1_280_mAU', 'UV_2_260_mAU'])]

    snr_values = []
    for col in ['UV_1_280_mAU', 'UV_2_260_mAU']:
        signal = baseline_data[col].values
        if len(signal) > 0:
            std_signal = np.std(signal)
            median_signal = np.median(signal)
            mad_signal = np.median(np.abs(signal - median_signal))
            if mad_signal > 0:
                snr = std_signal / mad_signal
            else:
                snr = 0.0
            snr_values.append(snr)
    return np.mean(snr_values) if snr_values else 0.0

df_filtered['avg_snr'] = df_filtered.groupby('run')['avg_snr'].apply(
    calculate_snr_group)

# Generate Output
lines = []
lines.append("## Data Integrity Report\n")

lines.append(f"### 1. Rows Filtered (Negative Volume)\n")
lines.append(f"Total rows removed due to #volume_ml# < 0: **{
    negative_volume_count}**\n")

lines.append("### 2. Summary Statistics (Filtered Dataset)\n")
stats_cols = ['UV_1_280_mAU', 'UV_2_260_mAU', 'Cond_mS_cm']
summary_df = df_filtered[stats_cols].agg(['mean', 'std', 'min', 'max'])
summary_df.columns = ['Mean', 'Std', 'Min', 'Max']
lines.append(summary_df.to_markdown(index=False))
lines.append("")

lines.append("### 3. Average SNR by Run\n")
snr_summary = df_filtered.groupby('run')['avg_snr'].mean().reset_index()
snr_summary.columns = ['run', 'Average_SNR']
lines.append(snr_summary.to_markdown(index=False))

with open(output_md_path, 'w') as f:
    f.write('\n'.join(lines))
```

```
    print(f"Report␣generated␣at␣{output_md_path}")
###
```

Listing 1: Code_1

```
###python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

def Code_Updates():
    # Load data
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter for required variables and exclude rows with missing pH_ml
    required_vars = ['run_no', 'UV_1_280_mAU', 'UV_2_260_mAU', 'System_flow_ml_min
        ', 'Sample_flow_ml_min', 'pH_ml']
    df = df[required_vars]
    df = df.dropna(subset=['pH_ml'])

    # Define normalization functions
    def static_normalization(group):
        mean_uv = group[['UV_1_280_mAU', 'UV_2_260_mAU']].mean(axis=1)
        return group - mean_uv

    def dynamic_normalization(group):
        flow_mean = group['System_flow_ml_min'].mean()
        if flow_mean == 0:
            return group
        return group / flow_mean

    # Apply normalizations using groupby to preserve structure
    df_static = df.groupby('run_no').apply(static_normalization).reset_index(drop=
        True)
    df_dynamic = df.groupby('run_no').apply(dynamic_normalization).reset_index(
        drop=True)

    # Check if df_dynamic is not empty before proceeding
    if df_dynamic.empty:
        raise ValueError("df_dynamic␣is␣empty.␣Cannot␣proceed␣with␣analysis.")

    # Define representative run selection
    # Find run_no with highest variance in dynamic normalization for
        representative plot
    rep_run = df_dynamic.loc[df_dynamic.groupby('run_no')['variance_iqr'].idxmax()
        ]
    rep_run_no = rep_run['run_no'].values[0]

    # Define rep_static and rep_dynamic by selecting the representative run
    rep_static = df_static[df_static['run_no'] == rep_run_no]
    rep_dynamic = df_dynamic[df_dynamic['run_no'] == rep_run_no]

    # Calculate variance (IQR) for each run_no using vectorized operations
    def calculate_iqr_variance_vectorized(group):
        uv_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']
        variances = []
        for col in uv_cols:
            q1 = group[col].quantile(0.25)
            q3 = group[col].quantile(0.75)
```

```
            variances.append(q3 - q1)
        return np.mean(variances)

    # Use agg with lambda for vectorized calculation
    df_static['variance_iqr'] = df_static.groupby('run_no')[['UV_1_280_mAU', '
        UV_2_260_mAU']].agg(lambda x: (x.quantile(0.75) - x.quantile(0.25)).mean())
    df_dynamic['variance_iqr'] = df_dynamic.groupby('run_no')[['UV_1_280_mAU', '
        UV_2_260_mAU']].agg(lambda x: (x.quantile(0.75) - x.quantile(0.25)).mean())

    # Create figure
    fig, axs = plt.subplots(1, 2, figsize=(14, 6))

    # Subplot 1: Boxplot comparing variance
    df_static['run_no'] = df_static['run_no'].astype(str)
    df_dynamic['run_no'] = df_dynamic['run_no'].astype(str)

    df_static['variance_iqr'] = df_static['variance_iqr'].astype(float)
    df_dynamic['variance_iqr'] = df_dynamic['variance_iqr'].astype(float)

    ax1 = axs[0]
    df_static.boxplot(column='variance_iqr', by='run_no', ax=ax1)
    ax1.set_title('Variance␣(IQR)␣by␣Run␣No', fontsize=12)
    ax1.set_ylabel('Variance␣(IQR)', fontsize=10)
    ax1.grid(axis='y', linestyle='--', alpha=0.7)

    # Subplot 2: Line plot for representative run
    ax2 = axs[1]

    ax2.plot(rep_static.index, rep_static['UV_1_280_mAU'], label='Static␣
        Normalized', linestyle='--')
    ax2.plot(rep_dynamic.index, rep_dynamic['UV_1_280_mAU'], label='Dynamic␣
        Normalized', linestyle='-.')

    ax2.set_title(f'UV␣Signal␣Comparison␣for␣Run␣{rep_run_no}', fontsize=12)
    ax2.set_xlabel('Index', fontsize=10)
    ax2.set_ylabel('mAU', fontsize=10)
    ax2.legend()
    ax2.grid(True, linestyle='--', alpha=0.7)

    plt.tight_layout()
    plt.savefig('/workspace/normalization_comparison.png', dpi=300, bbox_inches='
        tight')
    plt.close()
###
```

Listing 2: Code_2

```
import pandas as pd
import numpy as np
from scipy import stats

# Load the data
df = pd.read_csv('/inputs/chromatography_combined.csv')

# Debug: Print column names to verify data structure
print("Columns␣in␣loaded␣dataframe:", list(df.columns))

# Filter relevant columns and data
# Adjust column name based on actual data (e.g., 'Conc_B_%' or 'Conc_B')
```

```
data = df[['Conc_B_%', 'UV_1_280_mAU', 'run_no']].copy()

# Drop rows where Conc_B_% is missing or zero to ensure valid calibration points
data = data.dropna(subset=['Conc_B_%'])
data = data[data['Conc_B_%'] > 0]

# Drop rows where run_no is missing
data = data.dropna(subset=['run_no'])

# If run_no is still missing after dropping, use run as the grouping key
if 'run_no' not in data.columns or data['run_no'].isna().all():
    grouping_key = 'run'
else:
    grouping_key = 'run_no'

# Perform grouping with the validated key
run_data = data.groupby(grouping_key)['UV_1_280_mAU'].mean().reset_index()
run_data.columns = ['run_no', 'mean_UV']

# Merge back to keep only the runs with concentration data
calibration_data = run_data.merge(data[['run_no', 'Conc_B_%']], on='run_no', how='
    inner')

# Perform linear regression
x = calibration_data['Conc_B_%']
y = calibration_data['mean_UV']

slope, intercept, r_value, p_value, std_err = stats.linregress(x, y)

# Calculate R-squared
r_squared = r_value ** 2

# Create the report content
report_content = f"""# Calibration Curve Report

## Regression Equation
y = {slope:.6f}x + {intercept:.6f}

## Model Statistics
- R-squared: {r_squared:.6f}
- Standard Error of the Estimate: {std_err:.6f}
- P-value: {p_value:.6f}

## Conclusion
The relationship between UV signal and concentration is statistically significant
    (p-value < 0.05) and suitable for proxy quantification.
"""

# Append to the markdown file
with open('/workspace/calibration_report.md', 'a') as f:
    f.write(report_content)
```

Listing 3: Code_3